

## Search :

- API Scopes
- Rate Limiting
- CORS Configuration
- API Security

- **1-API Scopes :**

- **Scopes** هي صلاحيات (Permissions) يتم إعطاؤها للـ Access Token لتحديد ما يستطيع المستخدم فعله داخل الـ API.
- بدلاً من أن يكون الـ Token لديه صلاحية الوصول إلى كل شيء، يمكن تقييده بمجموعة محددة من الصلاحيات.
- مستخدم عادي يستطيع قراءة البيانات فقط.
- مدير (Admin) يستطيع القراءة والإضافة والتعديل والحذف.

بدلاً من إنشاء Token بدون صلاحيات: token = \$  
;\$user->createToken('api-token')->plainTextToken

- يمكن إنشاء Token مع Scopes:
- \$token = \$user->createToken( 'api-token', ['posts:read', 'posts:create']->plainTextToken;

هنا الـ Token يستطيع:

- قراءة المقالات. إنشاء مقالات. لكنه لا يستطيع حذفها.

## الفوائد

- زيادة الأمان.
- إعطاء أقل قدر من الصلاحيات (Least Privilege).
- إمكانية إنشاء Tokens مختلفة لنفس المستخدم

## 2- Rate Limiting:

هو تحديد عدد الطلبات (Requests) المسموح بها خلال فترة زمنية معينة.  
الهدف منه:

- منع الهجمات. منع الـ Spam. حماية الخادم. حماية Login.
- حماية قواعد البيانات. تقليل الضغط على السيرفر.
- تحسين الأداء. حماية الـ API من إساءة الاستخدام.

## لماذا نستخدمه؟

- منع Brute Force. حماية Login API. منع إساءة استخدام الـ API. تقليل الضغط على السيرفر.
- بدون Rate Limiting يمكن لمهاجم أن يرسل آلاف الطلبات في الثانية مما يؤدي إلى:
  - استهلاك موارد السيرفر. بطء التطبيق. توقف الخدمة

## 3-CORS Configuration :

CORS اختصار لـ:

### Cross-Origin Resource Sharing

وهو نظام أمني تطبقه المتصفحات للتحكم في إمكانية وصول موقع إلكتروني إلى API موجود على Domain أو Origin مختلف.

يتكون الـ Origin من:

● Protocol -Domain- Port

الفرونت : <http://localhost:5173> , الباك : <http://127.0.0.1:8000>

رغم أن كلاهما يعمل على نفس الجهاز، إلا أنهما يعتبران Origin مختلفاً لأن العنوان والمنفذ مختلفان.

ماذا يحدث؟

عندما يحاول React أو Flutter Web أو Angular استدعاء API موجود على Origin آخر، يقوم المتصفح بإرسال طلب.

إذا لم يسمح الـ Backend بذلك، يمنع المتصفح الاستجابة ويظهر الخطأ: Blocked by CORS Policy

أهم الإعدادات

**Allowed\_origins** من يستطيع الاتصال.

**Allowed\_methods** الطرق المسموح بها -Get- post- delete put

**Allowed\_header** الهيدرز المسموح بها. Content-Type -Accept Authorization

Preflight Request قبل تنفيذ بعض الطلبات مثل: put-patch-delete

يرسل المتصفح طلبًا من نوع: OPTIONS ليسأل السيرفر: هل تسمح بهذا الطلب؟ إذا وافق السيرفر يتم إرسال الطلب الحقيقي.

## CORS فوائد

- حماية المستخدم.
- منع المواقع الضارة من الوصول إلى بيانات المستخدم.
- التحكم في المواقع المسموح لها باستخدام الـ API.

## 4. API Security

هي مجموعة من الممارسات والتقنيات المستخدمة لحماية الـ API من:

- الاختراق.
- سرقة البيانات.
- التلاعب بالطلبات.
- الاستخدام غير المصرح به.

### ● أهم عناصر API Security

- Authentication : التحقق من هوية المستخدم.
- مثل: OAuth - JWT - Bearer Token - Laravel Sanctum
- Authorization : بعد التأكد من هوية المستخدم يتم التحقق مما إذا كان يمتلك صلاحية تنفيذ العملية المطلوبة.
- مثال: المستخدم العادي لا يستطيع حذف المستخدمين.
- Input Validation : منع إدخال بيانات غير صحيحة.
- Password Hashing : عدم تخزين كلمات المرور كنص عادي.
- HTTPS : تشفير جميع البيانات أثناء انتقالها
- SQL Injection Protection :
- Laravel Eloquent يستخدم Prepared Statements ،
- Rate Limiting : منع إرسال عدد كبير من الطلبات خلال فترة قصيرة

- API Scopes : إعطاء كل مستخدم أو كل Token أقل قدر من الصلاحيات المطلوبة.
  - Logging & Monitoring : تسجيل جميع الأخطاء والأنشطة المهمة حتى يمكن اكتشاف المشكلات أو محاولات الاختراق وتحليلها لاحقًا.
  -
- 

## أفضل الممارسات لتأمين API

- استخدام HTTPS دائماً.
  - استخدام Authentication قوي مثل Sanctum أو OAuth.
  - تطبيق Authorization على جميع العمليات الحساسة.
  - التحقق من جميع المدخلات.
  - استخدام Rate Limiting خاصة لمسارات تسجيل الدخول.
  - تقييد صلاحيات الـ Tokens باستخدام Scopes.
  - عدم إرجاع معلومات حساسة في الاستجابات.
  - تسجيل الأخطاء والأنشطة المهمة.
  - تحديث Laravel والحزم باستمرار لسد الثغرات الأمنية.
- باتباع هذه الممارسات يصبح الـ API أكثر أماناً وقادرًا على التعامل مع التطبيقات الحديثة مثل تطبيقات الويب والهواتف المحمولة بطريقة موثوقة.